

Endpoint Detection and Response

EDRdog

엔드포인트의 행위 로그로 공격을 탐지하고,
사람이 확인한 뒤 그 프로세스를 종료하는 EDR

단국대학교 소프트웨어학과 졸업작품

김동현 32210492

백신은 파일이 무엇인지 보고,
EDR은 파일이 무엇을 했는지 본다

백신

이 파일이 악성코드 목록에 있나?

알려진 악성코드 지문과 대조한다.
지문이 없는 새 공격은 그대로 통과한다.

EDR

이 파일이 방금 외부에서 받아온 것이고, 받자마자 실행됐나?

무엇을 실행했고, 어디에 접속했고, 어떤 순서였는지 본다.
하나씩은 정상인 행동의 조합을 잡아낸다.

01
파일 다운로드
정상

→

02
파일 실행
정상

→

03
받아온 그 파일을 실행
의심

하나씩 보면 정상이고, 순서로 이어 보면 위협이다.

목표: 수집부터 판정, 대응까지 한 흐름으로 잇는다

01
수집
엔드포인트의 행위 로그
프로세스, 스크립트, 네트워크, 파일, DNS, TLS

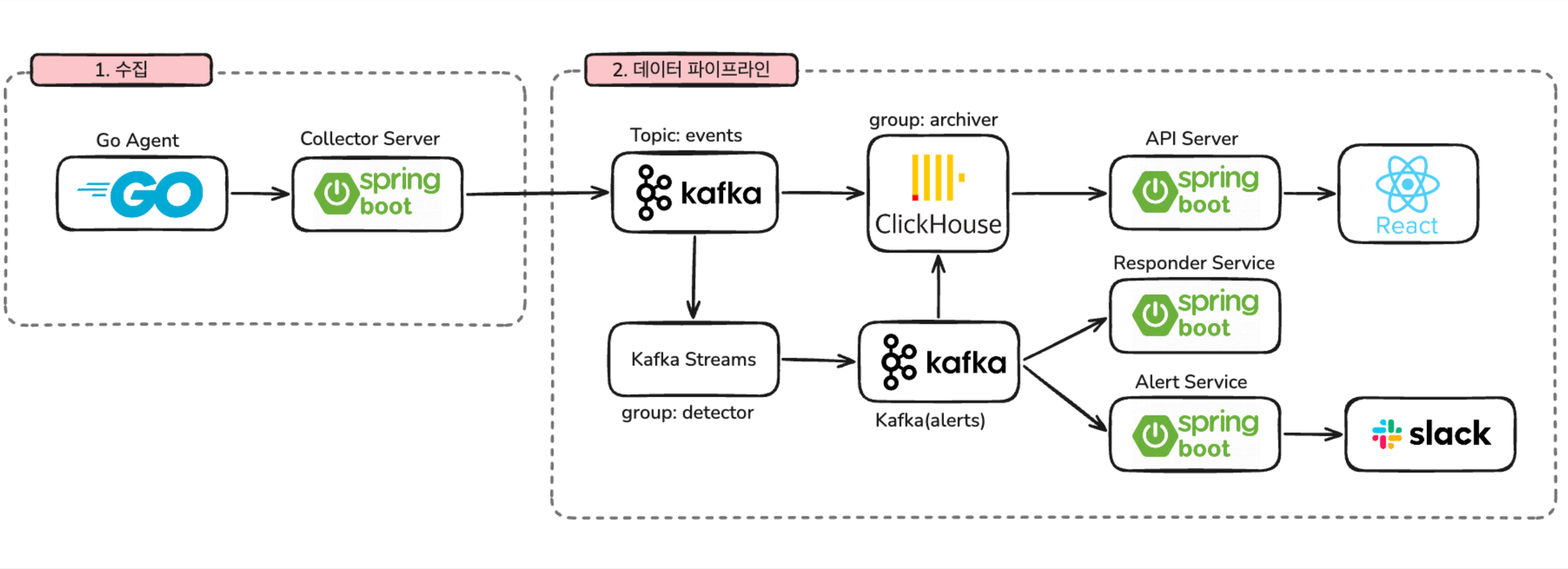
02
판정
여러 이벤트의 순서와 이벤트 하나를
규칙으로 판정하고 MITRE ATT&CK에 매핑

03
대응
Slack으로 알리고,
사람이 확인한 뒤 해당 프로세스를 종료

대상 기기	macOS, Windows 엔드포인트(Windows는 ARM64 VM에서 확인). Linux 등 그 외 플랫폼은 대상이 아니다.
대응 수단	프로세스 종료 하나. 자동 실행 없이 사람이 확인한 뒤 실행한다.
사용자	보안 담당자(admin), 조회 전용 사용자(viewer), 로그인 없이 둘러보는 데모 방문자
조직 구분	조직(tenant)마다 데이터와 명령을 나눈다. 조직은 기기가 아니라 서버가 정한다.

에이전트 하나, 서버 서비스 여섯, Kafka 토픽 둘

서비스 사이는 Kafka 토픽을 거치는 것이 기본이다. 토픽 수는 Streams 내부 토픽을 뺀 값이다.



events 토픽

3파티션, 키 host, Protobuf

alerts 토픽

3파티션, JSON

저장

ClickHouse는 이벤트와 판정, MySQL은 계정, 명령 큐, 처리 상태

자체 Go 에이전트가 프로세스와 네트워크를 함께 본다

왜 직접 만들었나

이전 구조는 **osquery와 Zeek** 이원 구조였다.

osquery는 소켓의 주인을 모르고, **Zeek는 PID를 모른다.**

다운로드 후 실행 판정은 **두 관점을 이어야** 성립한다.
Zeek는 Windows도 지원하지 않는다.

주기 조회 대신 **이벤트 구독**. 공격 도구는 짧게 실행되고 사라진다. 폴링은 macOS 네트워크 한 곳뿐이다.

운영체제별 수집 방식

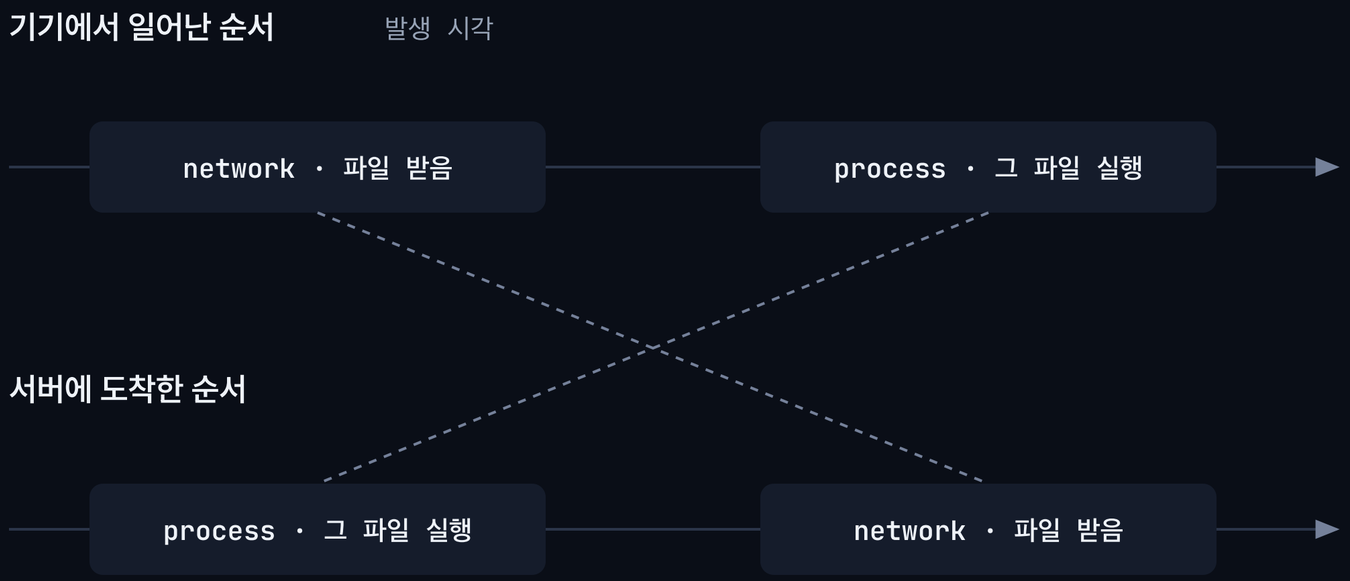
대상	macOS	Windows
프로세스 실행	eslogger exec (EndpointSecurity)	ETW Kernel-Process + 명령행 보강
파일	eslogger create, rename, unlink	ETW Kernel-File
네트워크 연결	libproc 소켓 스냅샷 폴링	ETW Kernel-Network
DNS	패킷 캡처	ETW DNS-Client
TLS SNI	패킷 캡처	pktmon 패킷 캡처

이벤트 종류 `process` `script` `network` `file` `dns` `l7` 최대 500건 배치, 기본 5초 주기, gzip 압축, 서버 인증서 고정

Windows는 Windows 11 ARM64 VM에서 다섯 종 수집을 확인했다(2026-11-11). x64는 센서 구동까지만 확인했다.

도착 순서가 아니라, 기기에서 일어난 시각으로 판정한다

host마다 최근 이벤트를 상태 저장소에 들고, 새 이벤트가 올 때마다 5분 윈도우 안에서 대조한다.



실행 이벤트가 네트워크 관측보다 먼저 도착했다.

도착 순서로 보면 다운로드 후 실행(R2)이 성립하지 않았다.

그래서 바꾼 것

01 판정 기준을 도착 순서에서 **엔드포인트가 찍은 발생 시각**으로 바꿨다.

02 버퍼는 새 이벤트보다 **앞선 것과 뒤선 것을 양쪽으로** 본다.

03 파티션 키와 워터마크를 **host 단위**로 둔다. 다른 기기의 이벤트 때문에 늦은 이벤트로 밀리지 않는다.

순서는 코드 규칙으로, 이벤트 하나는 데이터 규칙으로 판정한다

코드 규칙

Java 코드로 쓴 규칙. R1, R2는 여러 이벤트의 순서로 판정한다.

R1	오피스 문서가 셸을 띄움 순서 HIGH	T1059
R2	외부에서 받아온 파일을 실행 순서 CRITICAL	T1105 · T1204
R3	임시, 다운로드 경로에서 스크립트 실행	T1059
R4	자동실행 경로에 파일 생성	T1547
R5	낡은 TLS(SSL 3.0, TLS 1.0, 1.1)로 맺은 연결	T1573

데이터 규칙

SigmaHQ 공개 규칙을 에이전트가 보내는 필드에 맞게 옮긴 Sigma 부분집합 YAML

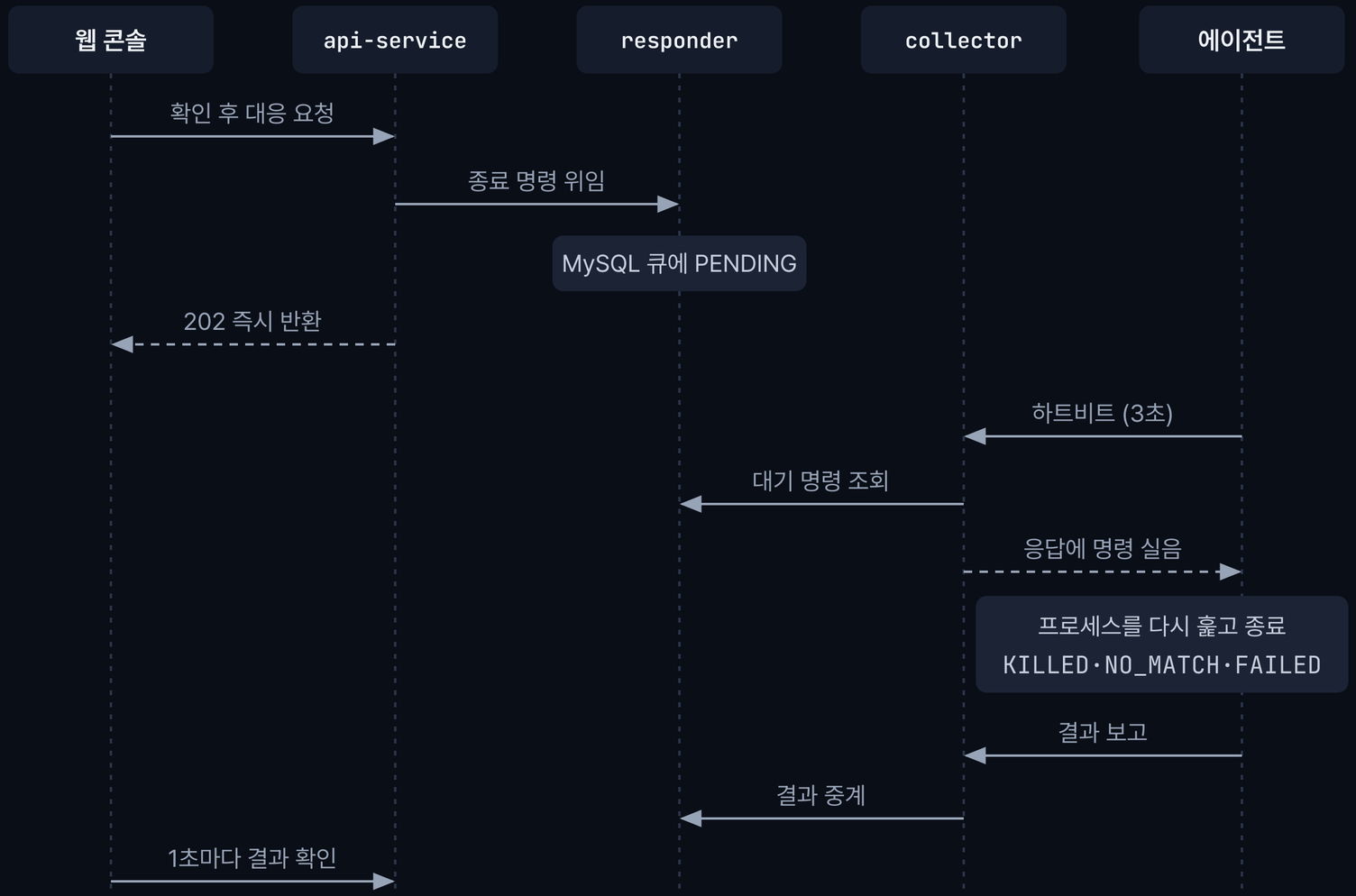
Windows 명령행	LOLBin 다운로드, 자격증명 덤프, 랜섬웨어 준비, 방어 무력화
macOS 명령행	권한 상승, 지속성, 방어 무력화
DNS	Tor, 파일 공유
파일	mimikatz, lsass 덤프 흔적

30초마다 다시 읽어 재배포 없이 반영한다. 운영은 ConfigMap으로 교체한다.

깨진 파일은 이전 규칙을 유지한다. 규칙 파일 때문에 탐지가 통째로 꺼지는 경로는 없다.

종료 권고는 HIGH 이상에만 붙는다. 데이터 규칙은 명백히 악성인 것만 HIGH로 두고 나머지는 알림만 보낸다.
데이터 규칙은 R2(CRITICAL)를 가리지 않는다.

사람이 확인하면, 다음 하트비트에 종료 명령이 내려간다



안전장치

명령 종류는 `kill_process` 하나. 스크립트를 내려보내지 않는다.

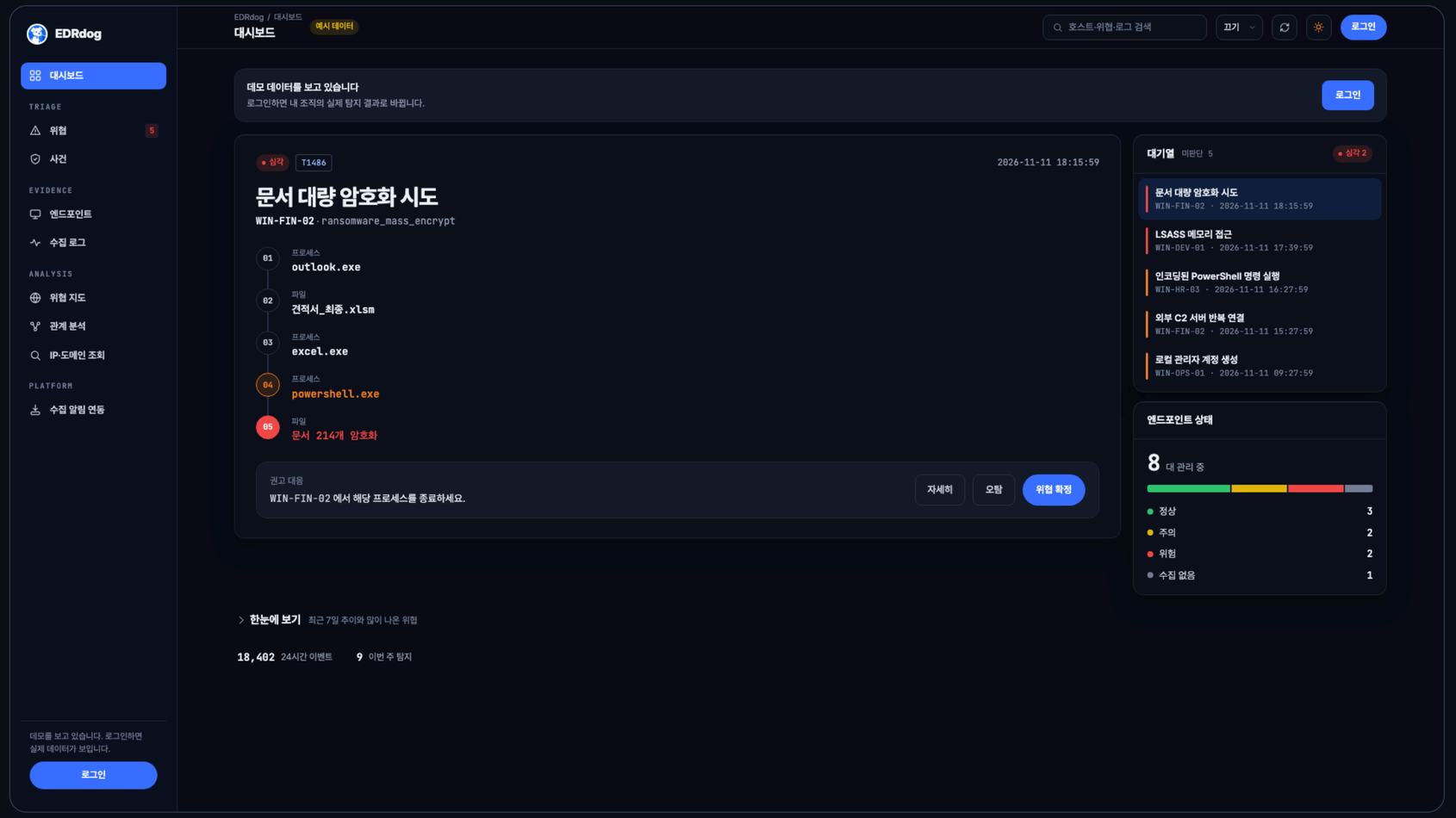
종료 대상은 요청 본문이 아니라 알림에서 꺼낸다.

자기 자신과 PID 1은 종료하지 않는다.

같은 명령을 다시 받으면 실행하지 않고 저장된 결과만 보고한다.

서버가 기기에 접속하지 않으므로, 기기에 포트를 열 필요가 없다.

대시보드는 가장 급한 미판단 위협 하나를 먼저 보여준다



화면의 숫자, 기기, 위협은 데모 예시 데이터다.

한 카드에서 판단

세로 공격 흐름, 권고 대응, 판정 버튼을 한 카드에 모았다.

옆에는 대기열

판단을 기다리는 위협과 엔드포인트 상태를 함께 둔다.

색은 심각도에만

바탕은 차분하게 두고 위험한 것만 색으로 드러낸다.

로그인 없이 체험

처음 온 사람도
데모 데이터로 둘러볼 수 있다.

사건은 알림을 프로세스 계보로 묶어 한 번에 판단한다



화면의 숫자, 기기, 위협은 데모 예시 데이터다.

계보로 묶기

같은 프로세스 계보에서 나온 알림을 사건 하나로 묶는다.

공격 경로 재구성

프로세스 계보, 증거 사슬, 타임라인을 한 화면에서 본다.

판단 뒤에도 유지

확정이나 오탐으로 판단한 뒤에도 계보가 남는다.

구조를 정한 네 가지 결정

결정	이유	버린 대안
ADR 001 · 002 **수집과 판정 사이에 Kafka, 판정은 Kafka Streams Processor API**	완충, 분기, 재처리(오프셋을 되돌려 재판정). 상태를 들고 순서를 판정하고, 라이브러리라 증설 때 파티션이 자동 재분배된다.	Flink, Spark: 별도 클러스터 운영 Streams DSL: 버퍼 판정 표현 불가
ADR 003 **중복은 EOS 대신 알림 단계에서 60초 억제**	문제는 중복 판정이 아니라 중복 알림이다. EOS는 처리량 저하와 지연 증가를 부른다. 억제 키는 수신 대상, host, 규칙.	EOS Redis: 파티셔닝 덕에 인스턴스 메모리로 충분
ADR 004 **조직은 기기가 보낸 값이 아니라 서버가 node_key로 정한다**	엔드포인트는 장악될 수 있다. node_key는 SHA-256 해시로만 저장한다.	BCrypt: 32바이트 난수라 사전공격 대상이 아니고, 매 요청 인증에 느리다
ADR 005 **쌓이는 로그는 ClickHouse, 사람이 바꾸는 상태는 MySQL**	불변 로그는 컬럼 스토어가, 판단 상태와 명령 큐처럼 사람이 바꾸는 상태는 트랜잭션이 맞다.	판정 기록을 MySQL에, 상태를 ClickHouse에 두는 안

알림은 Slack Incoming Webhook으로 보낸다. 기기 소유자의 개인 webhook이 있으면 그쪽으로, 없으면 조직 webhook으로 보낸다.

증상이 아니라 원인을 고친 세 가지 사례

증상	원인	해결
Windows ARM64 VM에서 ETW 센서가 시작 즉시 멈췄다.	golang-etw가 이벤트 ID 필터 버퍼를 붙잡아 두지 않아 Windows가 쓰레기 값을 읽었다. 크로스 빌드와 단위 테스트로는 드러나지 않았다.	라이브러리 사본을 저장소에 두고 버퍼를 힙에 두도록 패치했다. 센서가 전부 멈추면 종료 코드 1로 끝나 서비스 관리자가 다시 띄운다.
다른 조직에 같은 이름의 기기가 있으면 그 기기로 종료 명령이 갈 수 있었다. KISA 소프트웨어 보안약점 진단가이드(2021) 기준 코드 점검에서 발견	명령 큐가 조직 없이 host 이름만으로 명령을 찾았고, collector는 에이전트가 보낸 host를 그대로 썼고, 종료 대상을 요청 본문에서 받았다.	명령과 결과 보고를 조직과 host 쌍으로 묶고, host는 인증된 기기 값으로 덮어쓰고, 종료 대상은 알림에서만 꺼낸다.
detector의 Kafka Streams가 ERROR인데 health는 UP이었다.	기본 health는 프로세스가 살아 있는지만 본다.	Streams 상태를 health에 반영했다. 배포 뒤 점검(archiver 테이블 존재 확인)도 추가했다.

KISA EDR 평가 기준으로 직접 시험했다

KISA EDR 평가 항목 기준 자체 검증 공식 평가 아님

핵심 기능 (수집·탐지·대응)

20종

공격 시나리오를 재현해 실제로 잡는지 검증
자체 재현 10종 + Atomic Red Team 10종, 결함 수정·룰 보강 뒤 재시험 값

제품 자체 보안 (관리자 인증·암호 통신·변조 감지·감사 기록)

25/25

EDR 서비스가 공격 통로가 되지 않는지 검증
보안 항목 25개 통과

자동 테스트

Java 테스트 메서드 어노테이션 **1,319**개, Go 테스트 함수 **356**개, 퍼즈 함수 **3**개(코드에서 센 수)

CI는 main, dev 대상 PR에서만: Java 빌드, macOS 러너의 go test와 go vet. race 검출기는 로컬.

시간별 롤업(원본 7일 이후 보존이 목적)

host 하나의 7일 조회 **9ms → 4ms**

로컬 합성 데이터 200만 건, 7일치, host 50개

KISA 공격 시나리오와 시료는 공개되지 않아, 위협 보고서(Red Canary, Unit 42, CrowdStrike, Mandiant, Picus 등)에 공통으로 오르는 상위 기법으로 시나리오를 재현했다.

감사합니다

행위를 모으고, 순서로 판정하고,
사람이 확인한 뒤 프로세스를 종료한다.

데모	`https://edrdog.duckdns.org` 로그인 없이 데모 데이터로 둘러볼 수 있다
Backend	`github.com/EDRdog/Backend`
Frontend	`github.com/EDRdog/Frontend`